

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO2: *Apply Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies for decision-making tasks. (BL2, BL3)*

Assessment Tool Weightage: 30%

Dr G.A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A Application - Based Questions

Duration: 2Hrs

1. Design and develop a Reinforcement Learning framework a ride-sharing company uses Reinforcement Learning to match drivers with passengers efficiently. Apply RL concepts to define the state space, action space, and reward function to minimize waiting time and improve service quality. **(5 Marks)**
2. An e-learning platform uses Reinforcement Learning to recommend personalized study materials. Recall how exploration and exploitation strategies help balance familiar and new content recommendations. **(5 Marks)**
3. Design and develop a Reinforcement Learning framework a robotic vacuum cleaner uses Reinforcement Learning to clean efficiently while avoiding obstacles. Apply the concept of policy and explain how the value function improves performance over time. **(5 Marks)**
4. A cryptocurrency trading system uses Reinforcement Learning to decide buy, sell, or hold actions. Outline the challenges in designing reward functions and explain how poor reward design can affect performance. **(5 Marks)**
5. Develop a smart grid system uses Reinforcement Learning to manage electricity distribution. Create an RL model by defining state space, action space, and reward mechanism. **(5 Marks)**

6. A healthcare monitoring system uses Reinforcement Learning to adjust medication dosages. Apply RL concepts to define states, actions, and rewards, and explain how learning improves outcomes. **(5 Marks)**

7. Design a model drone delivery system uses Reinforcement Learning for navigation. Recall how exploration and exploitation strategies influence route optimization. **(5 Marks)**

8. **Design and develop** a Reinforcement Learning framework a manufacturing robot uses Reinforcement Learning to optimize assembly operations. Apply the concept of policy and explain how value functions help improve efficiency. **(5 Marks)**

9. An online advertising system uses Reinforcement Learning to maximize click-through rates. Outline reward design challenges and explain their impact on system performance. **(5 Marks)**

10. A smart irrigation system uses Reinforcement Learning to optimize water usage. Create an RL framework by defining states, actions, and reward function. **(5 Marks)**

Code of Conduct:

*I POLIDAS BHARATH___(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

bharath

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

1. Reinforcement Learning Framework for Ride-Sharing Driver-Passenger Matching

A ride-sharing company can use Reinforcement Learning (RL) to match passengers with suitable drivers and reduce passenger waiting time.

State Space

The state represents the current situation of the ride-sharing system. It may include:

- Location of available drivers.
- Location of passengers.
- Number of waiting passengers.
- Number of available drivers.
- Estimated travel time and distance.
- Traffic conditions.
- Driver availability and status.

Action Space

The RL agent can perform actions such as:

- Assign a specific driver to a passenger.
- Select an alternative driver.
- Keep a driver available for future requests.
- Reposition idle drivers to high-demand areas.

Reward Function

The reward should encourage efficient matching:

- Positive reward for quick passenger-driver matching.
- Positive reward for high driver utilization.
- Negative reward for long passenger waiting time.
- Negative reward for long pickup distances.
- Negative reward for cancelled rides.

A possible reward function is:

$$\text{Reward} = - \text{Waiting Time} - \text{Pickup Distance} + \text{Successful Match Bonus}$$

Conclusion

The RL agent learns the best driver-passenger matching policy through repeated interactions. Over time, it minimizes waiting time, improves driver utilization, and increases customer satisfaction.

2. Exploration and Exploitation in Personalized E-Learning Recommendations

An e-learning platform can use Reinforcement Learning to recommend personalized learning materials such as videos, articles, quizzes, assignments, and interactive exercises. The main challenge is deciding whether to recommend content that the student already likes or introduce new content.

This is addressed using **exploration and exploitation**.

Exploration

Exploration means trying new learning materials to discover whether they are useful for the student. For example, suppose a student usually watches programming videos. The platform may explore by recommending:

- An interactive coding exercise.
- A programming quiz.
- A new tutorial.
- A simulation-based learning activity.

The system observes whether the student watches the content, completes it, and performs well in assessments.

Exploitation

Exploitation means recommending learning materials that are already known to work well for the student.

For example, if a student frequently completes Python videos and performs well after watching them, the platform may continue recommending Python videos.

Balancing Exploration and Exploitation

If the platform only exploits, the student may repeatedly receive similar content and miss new useful resources.

If the platform only explores, the student may receive irrelevant or difficult materials. Therefore, the RL system must maintain a balance.

ϵ -Greedy Strategy

One common approach is the **ϵ -greedy strategy**.

- With probability $1 - \epsilon$, the system recommends the best-known learning material.
- With probability ϵ , the system recommends new or less-explored content.

For example, if $\epsilon = 0.2$:

- 80% of the time → Recommend familiar and successful content.
- 20% of the time → Explore new content.

Reward

The system can receive rewards based on:

- Content completion.
- Quiz scores.
- Time spent learning.
- Student engagement.
- Improvement in performance.
- Student feedback.

A possible reward is:

Reward = Learning Improvement + Content Completion + Engagement

Conclusion

Exploration helps the system discover new resources, while exploitation uses previously successful recommendations. By balancing both, the e-learning platform can provide personalized learning experiences and continuously improve its recommendations.

3. Reinforcement Learning Framework for a Robotic Vacuum Cleaner

A robotic vacuum cleaner can use Reinforcement Learning to learn how to clean a house efficiently while avoiding furniture, walls, stairs, and other obstacles.

The **agent** is the robotic vacuum, and the **environment** is the house or room in which it operates.

State Space

The state describes the robot's current situation. It can include:

- Current location of the robot.
- Direction in which the robot is moving.
- Areas that have already been cleaned.
- Areas that remain uncleaned.
- Battery level.
- Distance from obstacles.
- Presence of walls and furniture.
- Amount of dirt detected.
- Distance from charging station.

For example, if the battery is low and the robot is far from the charging station, this represents a specific state.

Action Space

The robot can perform actions such as:

- Move forward.
- Move backward.
- Turn left.
- Turn right.
- Stop.
- Increase or decrease cleaning speed.
- Return to charging station.

Policy

A **policy** defines the action that the agent should take for a particular state.

For example:

- If dirt is detected → Move toward the dirt.
- If an obstacle is detected → Change direction.
- If the battery is low → Return to the charging station.
- If an area is already clean → Move toward an unclean area.

The policy can initially be inefficient, but it improves as the agent learns from experience.

Value Function

The **value function** estimates how good a particular state is in terms of future rewards.

For example:

- A state where the robot is near an unclean area with sufficient battery may have high value.
- A state where the robot is stuck near an obstacle may have low value.
- A state where the robot has low battery and is far from the charger may have low value.

The robot learns to prefer actions that lead to states with higher expected future rewards.

Reward Function

Positive rewards:

- Cleaning dirt.
- Covering a new area.
- Avoiding obstacles.
- Efficient battery usage.
- Returning safely to the charging station.

Negative rewards:

- Collision with obstacles.
- Repeatedly cleaning the same area.
- Wasting battery.
- Getting stuck.

Conclusion

The robotic vacuum gradually improves its policy using the value function. It learns which actions lead to better long-term outcomes. As training continues, the robot can clean more area, avoid obstacles, save battery, and reduce unnecessary movement.

4. Reward Function Challenges in Cryptocurrency Trading

A cryptocurrency trading system can use RL to decide whether to **Buy, Sell, or Hold** digital assets. The agent attempts to maximize returns while controlling risk.

The **agent** is the trading algorithm, and the **environment** is the cryptocurrency market.

State Space

The state may include:

- Current cryptocurrency price.
- Historical price data.
- Trading volume.
- Market volatility.
- Moving averages.
- Technical indicators.
- Current portfolio value.
- Available cash.
- Amount of cryptocurrency held.

Action Space

The agent can choose:

1. **Buy** – Purchase cryptocurrency.
2. **Sell** – Sell cryptocurrency.
3. **Hold** – Maintain the current position.

Reward Function

The reward function is one of the most important and difficult parts of the system.

A simple reward could be based on changes in portfolio value:

Reward = Current Portfolio Value – Previous Portfolio Value

However, this may not be sufficient.

Challenges

1. Profit vs. Risk

If the system only rewards high profits, it may take excessive risks.

2. Transaction Costs

Frequent buying and selling can create transaction fees. Ignoring these costs may encourage overtrading.

3. Market Volatility

Cryptocurrency prices can change rapidly, making the reward unstable.

4. Short-Term vs. Long-Term Performance

A decision that produces short-term profit may cause long-term losses.

5. Overtrading

Poor reward design may encourage the agent to make unnecessary trades.

Improved Reward

A better reward can be:

Reward = Portfolio Profit – Transaction Cost – Risk Penalty

The risk penalty can discourage extremely risky decisions.

Impact of Poor Reward Design

If rewards are poorly designed, the agent may:

- Overtrade.
- Take excessive risks.
- Focus only on short-term gains.
- Ignore transaction costs.
- Lose money during volatile market conditions.

Conclusion

A well-designed reward function must balance profitability, risk, transaction costs, and long-term performance. Proper reward design is essential for developing a stable and reliable RL trading system.

5. Reinforcement Learning Model for Smart Grid Electricity Distribution

A smart grid can use Reinforcement Learning to efficiently manage electricity generation, storage, and distribution. The objective is to balance electricity supply and demand while reducing energy costs and wastage.

RL Components

The **agent** is the intelligent energy management system.

The **environment** includes:

- Power generation units.
- Renewable energy sources.
- Battery storage.
- Consumers.
- Electricity demand.
- Weather conditions.

State Space

The state can contain:

- Current electricity demand.
- Current electricity generation.
- Battery charge level.
- Renewable energy availability.
- Solar and wind power generation.
- Electricity price.
- Time of day.
- Weather conditions.
- Grid load.

For example, during the afternoon, solar power availability may be high, while during the evening, electricity demand may increase.

Action Space

The RL agent can:

- Increase electricity generation.
- Decrease electricity generation.
- Charge batteries.
- Discharge batteries.
- Use renewable energy.
- Use conventional energy.
- Shift flexible electricity demand.
- Distribute electricity to different regions.

Reward Mechanism

The reward should encourage efficient and reliable power management.

Positive Rewards:

- Meeting electricity demand.
- Using renewable energy.
- Maintaining grid stability.
- Efficient battery usage.

Negative Rewards:

- Energy wastage.

- High operating costs.
- Power shortages.
- Excessive battery usage.
- Grid instability.

A possible reward function is:

Reward = Demand Satisfaction + Renewable Energy Usage – Energy Cost – Energy Wastage – Shortage Penalty

Conclusion

The RL-based smart grid can dynamically manage electricity resources. It can reduce operational costs, improve renewable energy utilization, minimize wastage, and maintain reliable electricity distribution.

6. Reinforcement Learning for Healthcare Medication Dosage

A healthcare monitoring system can use RL to assist in personalized treatment decisions by learning from patient health conditions and treatment responses. Because medication dosage is safety-critical, any real-world RL system must be validated clinically and operate under healthcare professional supervision.

State Space

The state may include:

- Patient's vital signs.
- Blood pressure.
- Blood glucose levels.
- Heart rate.
- Current medication dosage.
- Previous treatment response.
- Relevant laboratory results.
- Symptoms.
- Patient age and weight.
- Overall health condition.

Action Space

The system may recommend:

- Increase dosage.
- Decrease dosage.
- Maintain current dosage.
- Change medication timing.
- Request additional monitoring.

In a real healthcare environment, these actions would require appropriate clinical oversight.

Reward Function

The reward function should encourage safe and effective treatment.

Positive Reward:

- Patient health moves toward the target range.
- Symptoms improve.
- Stable vital signs.
- Successful treatment response.

Negative Reward:

- Adverse side effects.
- Health deterioration.

- Dangerous readings.
- Incorrect dosage decisions.

A possible conceptual reward is:

Reward = Health Improvement – Side Effect Penalty – Risk Penalty

Learning Process

The system observes the patient's state and evaluates the treatment response. It uses this information to improve its future decision-making policy.

For example, if a particular treatment consistently improves the patient's condition without adverse effects, the system may learn that this action has a high expected value for similar states.

Conclusion

RL can support personalized treatment optimization by learning from patient responses. However, healthcare applications require strict safety controls, clinical validation, ethical considerations, and human supervision before deployment

7. Reinforcement Learning for Drone Delivery Navigation

A drone delivery company can use RL to determine efficient routes for delivering packages safely and quickly.

The agent is the drone navigation system, while the environment includes airspace, weather, obstacles, buildings, and delivery locations.

State Space

The state can include:

- Current drone location.
- Destination location.
- Current altitude.
- Battery level.
- Wind speed.
- Wind direction.
- Weather conditions.
- Obstacles.
- Restricted airspace.
- Remaining delivery distance.

Action Space

The drone can:

- Move forward.
- Move backward.
- Turn left.
- Turn right.
- Increase altitude.
- Decrease altitude.
- Change speed.
- Select an alternative route.

Exploration

Exploration means trying new routes to discover better paths.

For example, the drone may try an alternative route that has not previously been used. If the route consumes less energy and takes less time, the agent receives a positive reward.

Exploitation

Exploitation means using a route that has already been proven to be efficient and safe.

For example, if Route A has consistently produced fast and safe deliveries, the drone may continue using Route A.

Reward Function

The reward may include:

- Positive reward for successful delivery.
- Positive reward for reaching the destination quickly.
- Negative reward for excessive energy consumption.
- Negative reward for collisions.
- Negative reward for route delays.

Conclusion

Exploration helps the drone discover new and potentially better routes, while exploitation ensures that reliable routes continue to be used. A good balance between the two can improve delivery time, safety, and battery efficiency.

8. Reinforcement Learning Framework for a Manufacturing Robot

A manufacturing robot can use RL to optimize assembly operations in a factory. The objective is to reduce assembly time, minimize errors, improve product quality, and increase production efficiency.

State Space

The state can include:

- Current position of the robot arm.
- Current assembly stage.
- Position of components.
- Availability of components.
- Tool currently being used.
- Machine status.
- Assembly time.
- Quality of the assembled product.
- Presence of errors.

Action Space

The robot can:

- Pick up a component.
- Move to a location.
- Place a component.
- Rotate a component.
- Change tools.
- Adjust movement speed.
- Correct an assembly error.

Policy

The policy determines the best action for each state.

For example:

State: Component is available and correctly positioned.

Action: Pick up the component and place it at the required assembly location.

If the component is incorrectly positioned, the policy may select an action to adjust or reposition it.

Value Function

The value function estimates the expected future reward of a state.

For example:

- A state with correctly positioned components may have a high value.
- A state with an assembly error may have a low value.
- A state requiring unnecessary tool changes may have a lower value.

The robot learns to select actions that lead to high-value states.

Reward Function

Positive rewards:

- Successful assembly.
- High-quality product.
- Fast completion.
- Correct component placement.

Negative rewards:

- Assembly errors.
- Component damage.
- Long completion time.
- Unnecessary movements.
- Tool changes.

Conclusion

The manufacturing robot learns an efficient assembly policy through repeated interactions. The value function helps it select actions that provide better long-term outcomes. This can reduce production time, improve quality, and increase manufacturing efficiency.

9. Reward Design Challenges in Online Advertising

An online advertising platform can use RL to select advertisements that are most relevant to users. The goal is usually to improve engagement and business outcomes while maintaining a good user experience.

State Space

The state may include:

- User interests.
- Previous advertisements viewed.
- Previous clicks.
- Browsing behavior.
- Current webpage.
- Device type.
- Time of day.
- Advertisement history.

Action Space

The agent can:

- Select an advertisement.
- Select an advertisement category.
- Decide advertisement placement.
- Decide when to show an advertisement.
- Choose between different advertisements.

Reward Function

A simple reward function may give a positive reward when a user clicks an advertisement. However, this approach has several challenges.

Reward Design Challenges

1. Clicks Do Not Always Mean Success

A user may click an advertisement but immediately leave the website. Therefore, clicks alone may not represent genuine engagement.

2. User Experience

Showing too many advertisements may increase short-term clicks but reduce user satisfaction.

3. Long-Term Engagement

The system should consider whether users continue using the platform over time.

4. Conversion Rate

A click may not result in a purchase or registration. Therefore, conversion can be a more meaningful reward.

5. Advertisement Fatigue

Showing the same advertisement repeatedly may reduce its effectiveness.

Improved Reward

A possible reward function is:

Reward = Click Reward + Conversion Reward + Engagement Reward – Ad Fatigue Penalty – User Experience Penalty

Impact of Poor Reward Design

Poor reward design may cause the RL agent to:

- Select misleading advertisements.
- Show too many advertisements.
- Optimize only for clicks.
- Reduce user satisfaction.
- Increase user abandonment.
- Focus on short-term results rather than long-term engagement.

Conclusion

The reward function should balance click-through rate, conversions, user satisfaction, and long-term engagement. A carefully designed reward system can improve advertising performance without damaging the user experience.

10. Reinforcement Learning Framework for Smart Irrigation

A smart irrigation system can use RL to determine when and how much water should be supplied to crops. The main objective is to maintain healthy crop growth while minimizing water consumption. The **agent** is the intelligent irrigation controller, and the **environment** consists of soil, crops, weather, water resources, and irrigation equipment.

State Space

The state may include:

- Soil moisture level.
- Air temperature.
- Humidity.
- Rainfall.
- Weather forecast.
- Crop growth stage.
- Water tank level.
- Water availability.
- Previous irrigation history.

For example, if the soil moisture is low and no rainfall is expected, the system may identify the current state as requiring irrigation.

Action Space

The agent can:

- Turn irrigation ON.
- Turn irrigation OFF.
- Increase water flow.
- Decrease water flow.
- Select irrigation duration.
- Select irrigation zones.

Reward Function

The reward should balance crop health and water conservation.

Positive Rewards:

- Maintaining optimal soil moisture.
- Healthy crop growth.
- Efficient water usage.
- Avoiding crop stress.

Negative Rewards:

- Excessive water consumption.
- Over-irrigation.
- Under-irrigation.
- Water wastage.
- Irrigating before expected rainfall.

A possible reward function is:

Reward = Crop Health Benefit – Water Consumption Penalty – Underwatering Penalty – Overwatering Penalty

Example

Suppose soil moisture is below the required level. The RL system activates irrigation for an appropriate duration. If rainfall is expected soon, the system may delay irrigation to conserve water.

Over time, the system learns the relationship between weather, soil moisture, irrigation, and crop health.

Conclusion

The RL-based smart irrigation system can automatically learn efficient watering strategies. It can reduce water wastage, maintain suitable soil moisture, support healthy crop growth, and improve the overall

efficiency of agricultural water management.